design pattern

类于类之间的关系

分类

继承（Generalization / Inheritance）

- "is-a"关系，子类继承父类的属性和方法，可扩展或重写。这是类之间耦合度最高的一种关系
- 在类图的画法上：从派生类指向基类的空心三角箭头

关联（Association）

一个类持有另一个类的指针或引用，作为成员变量存在

- 单向关联：A 知道 B，B 不知道 A
  - 在代码层面上：A类包含B类型的指针或者引用。
  - 在类图的画法上：从A指向B的直线箭头
- 双向关联：A 和 B 互相知道对方
  - 在代码层面上：使用的是指针或者引用。
  - 在类图的画法上：直接使用实心直线

聚合（Aggregation）

- 一种"has-a"关系，表现为整体与局部的关系, 但整体与部分之间生命周期独立。例如车（Car）和轮胎（Tire），车被销毁后轮胎仍可存在。
- 关联关系的一种特例。
- 在代码层面上：使用的是指针或者引用。
- 在类图的画法上：从局部指向整体的空心菱形箭头

组合（Composition）

- 更强的"has-a"关系，表现为整体与局部的关系, 整体与部分共存亡。例如人（Person）和心脏（Heart），人消失则心脏也失去意义。
- 组合是聚合的一种特殊形式。
- 在类图的画法上：从局部指向整体的实心菱形箭头

依赖（Dependency）

- 一种"A use B"的关系，这种关系是偶然的，临时的，并非固定的, 关系最弱
- 代码层面上
  - B作为A的成员函数参数；（或者B类引用、B类指针作为A的成员函数参数）
  - B作为A的成员函数的局部变量；
  - B作为A的成员函数的返回值；
  - A的成员函数调用B的静态方法
- 在类图的画法上：使用虚线的箭头，从A指向B

高内聚, 低耦合

高内聚、低耦合是软件设计（尤其是面向对象设计）的核心原则，目的是让代码更容易维护、扩展和复用

内聚

- 指一个模块（类、方法）内部各个部分之间的关联程度
- 一个模块只做一件事，而且所有内部元素都紧密协作完成这一件事

耦合

- 指不同模块之间的依赖程度
- 模块之间联系尽量少、弱。修改一个模块尽量不牵涉其他模块

举例

- 高内聚：每个成员专注于自己的角色（前台只接待、程序员只写代码）。
- 低耦合：不同角色之间只用规定的沟通方式（邮件、钉钉），而不是互相闯入对方工位乱翻。

耦合程度

依赖 < 关联 < 聚合 < 组合 < 继承

设计模式原则

设计模式是招式，而设计原则是心法

单一职责原则 (SRP)

- 定义　一个类只该有一个引起变化的原因。
- 通俗理解　一个类只干好一件事。
- 举例
  - 学生的职责任务就是学习, 只干学习这一件事, 不要打游戏
  - 手机, 摄像机
- 如果类承担过多职责，一个职责的变化可能会削弱或抑制该类完成其他职责的能力，导致意想不到的副作用。而且职责越多，代码越难理解、难测试、难复用。
- 案例　智能手机, 拆分成不同功能

开闭原则 (OCP)

- 定义　对扩展开放，对修改关闭。
- 通俗理解　新增功能时，尽量不去修改已有的稳定代码，而是通过增加新代码来实现
- 举例　电脑内存不够, 想给电脑加内存，你插根新内存条，而不是把主板焊点重画。
- 修改已有、经过测试且稳定运行的代码容易引入新bug。依靠扩展，可以将变化局部化，系统更稳定。
- 案例　计算器类

里氏替换原则 (LSP)

- 定义　子类对象必须能够完全替换掉父类对象，且程序行为不变。
- 通俗理解　父亲能做的事，儿子都能做，而且结果符合预期
- 案例　正方形长方形

接口隔离原则 (ISP)

- 定义
  - 不应强迫客户端依赖它们不使用的方法, 使用多个小的专门的接口，而不要使用一个大的总接口
  - c++中没有接口这种语法, 这里表示的是一个逻辑概念, 有两种理解
    - 一种是指一组方法的集合，这组方法由实现接口的类提供；
    - 另外一种是指C++语言中声明纯虚函数的抽象类(只有纯虚函数,没有数据成员)
- 通俗理解　别弄个大而全的"万能接口"，让调用者不得不实现它根本用不着的方法。应该把肥硕的接口切分成多个小而专注的接口
- 案例　鸟类, 鸵鸟

依赖倒置原则 (DIP)

- 定义　高层模块不应依赖低层模块，二者都应依赖其抽象；抽象不应依赖细节，细节应依赖抽象。
- 通俗理解　要依赖抽象（接口），不要依赖具体的实现类---->面向接口编程
- 程序中应用
  - 依赖倒置原则要求在程序代码中传递参数时或在关联关系中尽量引用层次高的抽象层类
    - 使用抽象类进行变量类型声明
    - 参数类型声明
    - 方法返回类型声明
  - 父类型接收子类型
- 案例　银行 职员

迪米特法则 (最少知识原则)

- 定义　一个对象应该对其他对象有尽可能少的了解，只和"直接朋友"通信，不和"陌生人"说话。
- 核心思想　降低耦合程度
- "直接朋友的分类"
  （1）当前对象本身（this）
  （2）以参数形式传入到当前对象方法中的对象
  （3）当前对象的成员子对象
  （4）如果当前对象的成员对象是一个集合，那么集合中的元素也都是朋友
  （5）当前对象的成员函数中创建的对象
- 案例　客户买房
- 也可以用pimpl模式理解

组合复用原则

- 定义　优先使用对象组合，而不是通过继承来达到复用的目的。
- 核心思想　在复用时要尽量使用组合/聚合关系（关联关系），少用继承 ----组合优于继承
- 案例　交通工具

设计模式

- 单例
- 代理

工厂设计模式

简单工厂(静态工厂)

- 提供一个专门的工厂类，在工厂类中做判断，根据传入的类型创造相应的产品，这样做能够将对象的创建和对象的使用分离开。当增加新的产品时，就需要修改工厂类。
- 优点　实现简单: 一个工厂类集中管理所有产品对象的创建逻辑，客户端无需关心具体类名
- 缺点
  - 违背OCP开闭原则
  - 扩展性差
- 适用场景　适合产品种类少且基本不变的场景

工厂方法

- 定义一个创建产品的抽象工厂接口，每个具体工厂负责创建一种具体产品。新增产品只需增加对应的具体工厂类，符合开闭原则
- 核心的工厂类不再负责产品的创建，而是演变为一个抽象工厂角色，仅负责定义具体工厂子类必须实现的接口
- 优点
  - 符合开闭原则：增加新产品只需新增对应的具体产品类和具体工厂类，无需修改已有工厂接口或客户端。
  - 单一职责：每个具体工厂只负责创建一种产品，职责清晰。
  - 易于扩展：产品层级结构（例如苹果、香蕉、橙子……）可以无限增加，不影响已有代码。
  - 支持多态创建：使用时依赖抽象工厂，运行时动态选择具体工厂，实现灵活的对象创建
- 缺点
  - 1、类的个数容易过多，增加复杂度
  - 2、增加了系统的抽象性和理解难度
  - 3、接口的传入参数类型需要一致
  - 4、抽象产品只能生产一种产品，不适合需要同时创建一系列相关产品的场景。此弊端可使用抽象工厂模式解决。
- 适用场景
  - 1.客户只知道创建产品的工厂名，而不知道具体的产品名。
  - 2. 产品种类经常变化，但每种产品的创建逻辑相对独立
  - 3.客户不关心创建产品的细节，只关心产品的品牌

抽象工厂

- 提供一个接口, 该接口负责创建一系列相关或相互依赖的产品对象（产品族: 同一品牌/风格下的一组相关产品）,无需指定它们具体的类。
- 优点
  - 1.保证产品族的一致性：同一个具体工厂生产的所有产品都属于同一风格/品牌（例如中国苹果 + 中国香蕉），确保搭配协调。
  - 2.增加产品族容易：新增一个产品族（例如日本风格）只需增加对应的具体工厂，无需修改现有工厂接口或已有具体工厂，符合开闭原则。
  - 3.高度解耦：客户端只依赖抽象工厂和抽象产品接口，不关心具体产品的实现细节。
  - 4.便于替换整个产品族：只需更换具体工厂实例，就能一键切换整套产品实现。
- 缺点
  - 1.增加产品等级结构困难：如果需要新增一种产品类型（例如橙子），必须修改抽象工厂接口（添加 createOrange()），进而修改所有已有的具体工厂类，违背开闭原则。
  - 2.系统结构复杂：涉及多个抽象接口、多个具体工厂、多个产品等级结构，类数量膨胀明显。
  - 3.扩展灵活性受限：适合产品等级结构相对稳定的场景，若产品类型频繁变化（如经常需要新增"橙子、葡萄…"），则维护成本很高。
- 适用场景
  - 产品等级结构固定，但产品族多变（例如不同操作系统的 GUI, 手机.平板 电视 电脑...）
  - 需要保证多个产品（例如 UI 中的按钮、文本框、滚动条）风格一致。

总结

- 想要一个工厂生产很多种产品 —— 简单工厂模式
- 想要生产较多的新产品 —— 工厂模式
- 生产产品时想要对产品的层级进行更细化的划分 —— 抽象工厂

观察者

- 定义　定义了对象之间一对多的依赖关系。当一个对象的状态发生变化时，所有依赖于它的对象都会得到通知并被自动更新。这种模式有时也被称作发布-订阅模式
- 适用场景　在软件系统中，经常需要多个对象对某个对象的状态变化做出响应
  - GUI 中点击按钮，多个界面元素需要更新
  - 气象站数据更新，多个显示面板（温度、湿度、气压）需要同步
  - Up主发视频, 粉丝收到更新通知
- 核心逻辑　主题（Subject），即被观察者，不关心有哪些观察者，只负责维护一个观察者列表并在变化时通知它们；观察者（Observer） 只需订阅主题，即可在通知到来时拉取或接收数据
- 标准结构
  - Subject（主题/抽象目标）　提供注册、注销观察者的接口，以及通知所有观察者的方法
  - ConcreteSubject（具体主题）　维护状态，当状态变化时触发通知
  - Observer（抽象观察者）　定义一个更新接口，用于接收通知。
  - ConcreteObserver（具体观察者）　实现更新接口